

Могућа и постојећа решења

Наивно решење

Опште решење реализације *Mediator* обрасца је приказано сликом Figure 1. Општа реализација обрасца подразумева увођење абстракције (интерфјес *Mediator*) које је асоцирана са конкретном имплементацијом логики *Mediator* објекта (*ConcreteMediator*). Приликом позива методе за примање одређеног захтева (нпр. *dispatch*), конкретна имплементација обрасца је у стању да констатује којој компоненти је неопходно делегирати даље извршавање логики (нпр *ConcreteColleague1*).

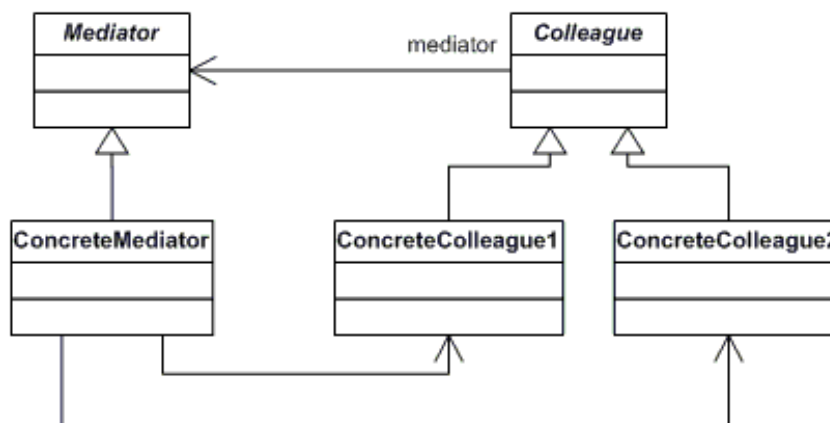


Figure 1: Дијаграм опште имплементације *Mediator* обрасца.

У општем случају, овакво решење је задовољавајуће за једноставније сценарије коришћења. Упркос томе, главни проблеми који се јављају при стварној имплементацији су:

- **Пораст величине посредничког објекта (енг. *God Object Problem*)** – Како систем расте и креирају се нове функционалности, јавља се потреба за проширењем кода посредничког објекта. Ово у најужем смислу обухвата креирање нових *Colleague* објеката и њихову регистрацију. У радним оквирима *Java* екосистема (попут *Spring* радног оквира), ово се манифестује кроз континуално инјектовање нових зависности унутар посредничког објекта.
- **Кршење принципа отворености/затворености (енг. *Open/Closed Principle*)** – Свако увођење нове функционалности или међусобне комуникације захтева директну модификацију постојеће класе *ConcreteMediator*, уместо њеног проширивања без измене постојећег кода.
- **Сложеност тестирања** – Због броја зависности које посреднички објекат акумулира, писање јединичних тестова за *ConcreteMediator* постаје компликовано, јер захтева конфигурисање великог броја лажних објеката (*mock* објеката).

Постојећа решења у *Java* екосистему

Потреба за сегрегацијом контролних токова и централизацијом комуникационе логике је у *Java* екосистемима произвела неколико познатих решења.

Axon Framework

Axon Framework је софтверски оквир намењен изградњи дистрибуираних *Java* система. Оквир се у великој мери ослања на архитектуру вођену догађајима (енг. *Event-Driven Architecture*).

У контексту *Mediator* обрасца, *Axon* елиминише проблем монолитног и преоптерећеног посредничког објекта тако што уводи децентрализоване магистрале порука (енг. *Message Buses*). Уместо једне конкретне класе која зна за све компоненте, систем се заснива на три кључна типа порука, за које постоје специјализовани посредници:

- **Магистрала команди (енг. *Command Bus*)** – Посредник задужен за рутирање команди (намера за променом стања система) до њихових специфичних обрађивача (@CommandHandler). Свака команда има тачно једног одредишног примаоца, чиме се постиже чиста сегрегација пословне логике.
- **Магистрала догађаја (енг. *Event Bus*)** – Након што се стање система измени, генерише се догађај који се путем ове магистрале дистрибуира до свих заинтересованих компоненти (@EventHandler). Компонента која је иницирала промену нема никакво сазнање о томе које све компоненте реагују на ту промену.
- **Магистрала упита (енг. *Query Bus*)** – Магистрала намењена рутирању упита за приказ података до одговарајућих обрађивача (@QueryHandler), подржавајући и концепте попут тачка-до-тачке, растрканих (енг. *scatter-gather*) или претплатничких упита.

Главна предност коју *Axon* доноси у односу на наивну имплементацију јесте динамичка регистрација компоненти, односно елиминација потребе за ручном регистрацијом и инјекцијом компоненти. Нови обрађивачи команди или догађаја се региструју путем анотација, без икакве потребе за модификацијом самог посредничког објекта. Међутим, највећа мана овог оквира је висока комплексност, велика потреба за савладавањем оквира и инфраструктурно оптерећење. Мане *Axon* оквира условљавају да је оквир често превише комплексно решење за стандардне пословне апликације.

PipelinR

PipelinR је *Java* библиотека примарно намењена за *Spring* радни оквир. Библиотека за циљ има да омогући чисту имплементацију команди и упита, без комплексне инфраструктуре какву захтевају већи радни оквири попут *Axon*-а. Дизајнирана је по узору на популарне библиотеке из других екосистема (као што је *MediatR* у *.NET* екосистему).

Уместо централног објекта који садржи референце ка свим сервисима, *PipelinR* уводи концепт цевовода (енг. *pipeline*) и заснива се на три основна градивна блока:

- **Команда (енг. *command*) или упит (енг. *query*)** – Представља једноставан објекат који описује намеру или захтев.
- **Обрађивач (енг. *Handler*)** – Компонента задужена за извршавање конкретне пословне логике везане за одређену команду.
- **Цевовод (енг. *Pipeline*)** – Сам посреднички објекат који прихвата команду и аутоматски проналази и покреће одговарајући *Handler*.

Библиотека не поседује уграђен механизам за аутоматско скенирање и регистрацију релеватних компоненти. Упркос томе, омогућава да се кроз *Spring* конфигурационе класе

прикупе сви обрађивачи из апликативног контекста и проследе централном посредничком објекту. Након што се ова почетна конфигурација постави, увођење нове функционалности захтева само креирање нових `Command` и `Handler` класа (означених са анотацијом `@Component`). *Spring* ће их аутоматски скенирати и доставити цевоводу, чиме се у потпуности елиминише потреба за ручном инјекцијом нових зависности у сам посреднички објекат и задовољава *Open/Closed* принцип.

Главна мана овог решења је што је библиотека ограничена искључиво на синхрону комуникацију по моделу “један на један” (једна команда – један обрађивач), те није погодна за системе који захтевају асинхронно раштркавање порука или архитектуру вођену догађајима са више претплатника.

Постојећа решења из других екосистема

MediatR у *C#*

MediatR је једна од најпопуларнијих и најшире прихваћених библиотека у *.NET* екосистему. Библиотека је постала стандард за имплементацију *Mediator* обрасца у овом окружењу. Својим именом и утицајем библиотека је поставила образац са деривате *Mediator* имплементације у другим програмским језицима.

MediatR се заснива на концепту раздвајања порука на команде/упите и њихове обрађиваче. Главни посреднички објекат прихвата поруку и динамички је рутира до одговарајућег обрађивача. Као софтверско решење, он нуди неколико кључних предности у решавању проблема наивне имплементације:

- **Аутоматско скенирање** – За разлику од својих деривата у другим језицима, *MediatR* долази са уграђеним екстензијама за *.NET* контејнер уметања зависности. То омогућава да се унутар конфигурације апликације позове само једна метода која аутоматски скенира склопове (енг. *assemblies*), проналази све дефинисане обрађиваче и региструје их. Тиме се у потпуности елиминише потреба за модификацијом посредничког објекта.
- **Разноврсност комуникационих модела** – Поред синхроног “један на један” рутирања, *MediatR* подржава и модел објављивања догађаја вишеструким претплатницима (нотификације), што му омогућава да функционише и као локална магистрала догађаја унутар једне апликације.
- **Цевоводи (енг. *Pipeline Behaviors*)** – Ова функција омогућава креирање пресретаца који омогућавају једноставно убацивање попречних функционалности (енг. *cross-cutting concerns*) попут валидације, логовања, кеширања и управљања трансакцијама дуж саме путање захтева, унапређујући тако сегрегацију пословне логике.

Опште мане имплементација у различитим екосистемима

Импликације везане за сегрегацију попречне логике

Потреба за сегрегацијом попречне логике (енгл. *cross-cutting concerns*) представља изазов у пројектовању софтвера. *C#* библиотека *MediatR* овај проблем решава путем експлицитно дефинисаних цевовода који омотавају крајњи обрађивач поруке. Редослед извршавања цевовода се јасно дефинише унутар *.NET* контејнера за инјекцију зависности, те је преглед тока извршавања транспарентан.

Са друге стране, *Java* екосистем се традиционално ослања на механизам Аспектно-оријентисаног програмирања (енгл. *Aspect-Oriented Programming* – *AOP*). Иако *AOP* прати

декларативну природу и устаљене конвенције *Java* програмског језика, овакво решење уноси неколико значајних недостатака:

- **Специфична и комплексна синтакса** – Дефинисање аспеката и тачака пресретања (енгл. *pointcuts*) захтева знање специфичног језика и правила која повећавају сложеност кода.
- **Отежано праћење тока извршавања** – Имплицитно окидање аспеката “у позадини” онемогућава лаку анализу кода, те чини праћење стварног пута извршавања компликованим.
- **Осетљивост на рефакторисање** – *AOP* пресретачи често ослањају на текстуалне шаблоне за путање пакета или имена класа. Било какво рефакторисање или реорганизација структуре пројекта може узроковати да се логика садржана у аспектима не извршава.
- **Децентрализација и фрагментација пословне логике** – Логика која подржава одређену функционалност бива физички измештена ван компоненте која је извршава, чиме се смањује прегледност и отежава разумевање контекста у коме се апликација извршава.

Архитектуралне импликације

Претходно наведене имплементације решавају недостатке наивног приступа. Упркос томе, оне често отварају нове изазове и импликације архитектуралне природе. Неки од проблема везаних за саму архитектуру система су:

- **Ризик од неисправне резолуције и преклапања обрађивача** – Уколико се грешком региструје више обрађивача за исти тип поруке, нарушава се стриктна “један-на-један” веза између поруке и њеног јединственог обрађивача. Оваква лоше постављена конфигурација се не уочава током компајлирања, већ се испољава искључиво у време извршавања апликације (енг. *runtime*). То резултује грешкама у пословној логици или бацањем изузетака. Додатно, неки оквири једноставно пребришу претходне обрађиваче и покрену само последњи регистровани.
- **Нарушавање принципа јединствене одговорности (енг. *Single Responsibility Principle*)** – Класа која има улогу обрађивача порука требало би да буде задужена за извршавање логике једног и само једног типа поруке. Међутим, у већини актуелних библиотека не постоји ограничење високог нивоа које би експлицитно забранило да једна класа бива задужена за обраду различитих порука. Дозвољавањем овакве праксе, класа може да акумулира вишеструке одговорности, чиме се директно принцип јединствене одговорности.
- **Проблем наслеђивања између класа порука** – Уколико један тип поруке (класа) наслеђује други, семантика и логика извршавања постају нејасне. Са становишта архитектуре, поставља се комплексно питање резолуције: да ли поруку изведеног (подређеног) типа треба обрадити искључиво путем обрађивача који је за њу експлицитно дефинисан, или је треба проследити и обрађивачу надтипа, или пак активирати оба у одређеном редоследу. Библиотеке често немају јединствен одговор на овај проблем, што уноси непредвидивост у рад система.
- **Заобилажење посредничког објекта директном инјекцијом компоненти** – Дизајн заснован на *Mediator* обрасцу захтева да се међусобна интеракција између компоненти спроводи искључиво преко њега. Међутим, у пракси је и даље технички дозвољено да се инстанцирају или директно инјектују компоненте којима руководи *Mediator* објекат. Експлицитно повезивање компоненти, чијом би комуникацијом примарно требало да руководи посредник, нарушава основни смисао његовог постојања посредничког објекта и смисла лабавог повезивања.

- **Непостојање обрађивача за конкретни тип поруке** – Већина имплементација омогућава да се порука пошаље кроз систем чак и када не постоји регистрован обрађивач за њу. Понашање система варира од оквира до оквира. Поједине библиотеке ће бацити изузетак и прекинути извршавање, док ће друге “прогутати” поруку без икаквог обавештења. Не постоји сврха за креирање објеката порука који немају конкретног пара у виду обрађивача.

Искуствене импликације

Многе имплементације *Mediator* обрасца су стриктно везане за одређени софтверски екосистем или радни оквир. Из погледа инжењера који развија апликацију, овакав дизајн ствара когнитивно оптерећење приликом транзиције на друге технологије. Уместо да се искључиво ослони на концепте дизајна, инжењер је приморан да учи инфраструктурне специфичности нове библиотеке. Ово нарушава сигурност у развоју и флексибилност инжењера. Имплементације често не подражавају универзалне архитектонске принципе, већ прате искључиво парадигму једног конкретног радног оквира.

Са друге стране, за решење које тежи пружању уједињеног корисничког и развојног искуства кроз различите радне оквире, намеће се потреба за стриктним поштовањем специфичних конвенција и идиома сваког појединачног окружења. Иако суштина остаје конзистентна, решење мора да се прилагоди устаљеним парадигмама конкретног радног оквира. Циљ прилагођавања је не нарушити конзистентност, архитектуру и постојеће концепте које диктира радни оквир.